Learning What to Change: From Visual Reasoning to Scientific Hypothesis Revision

Chimdi Walter Ndubuisi University of Missouri

Executable reasoning, failure diagnosis and generalization across semantic domains · REVIEW DRAFT, 18 October

The question

How can a system learn to reason about things it was never explicitly programmed to understand? Three parts:

1. **Representation:** can it induce the right entities from sparse data of a new kind?
2. **Rules:** can it discover the executable rules that act on them?
3. **Revision and transfer:** when it fails, can it tell *which part* to change, and does that procedure carry over to a domain whose meaning is different?

These questions were shaped in discussion with Toni Kazic.

Why one solved puzzle proves little

Several programs can fit every demonstration and still disagree on the test. On 60 exposed ARC-AGI-2 training tasks, CORA's object reasoner had demonstration-fitting programs that were **all wrong** on 26 tasks (with its frozen assets).

A failure can come from three different places, and each needs a different change:

- the **representation** (which objects exist);
- the **rule** (what happens to them);
- the **evidence** (the examples do not decide).

The wrong revision can break what already worked.

Where this sits

- **Popper** (Cropper & Morel): program induction that learns from failed hypotheses.
- **DreamCoder** (Ellis et al.), **LILO** (Grand et al.): growing reusable program libraries.
- **Model Discovery Agent** (Murphy): model inadequacy driving hypothesis revision and experiment design.
- Frontier neural reasoners are strong; they are comparisons, not straw men.

The narrower question here: choosing *which component* to revise, and showing that this choice beats an equally equipped system that does not use the diagnosis.

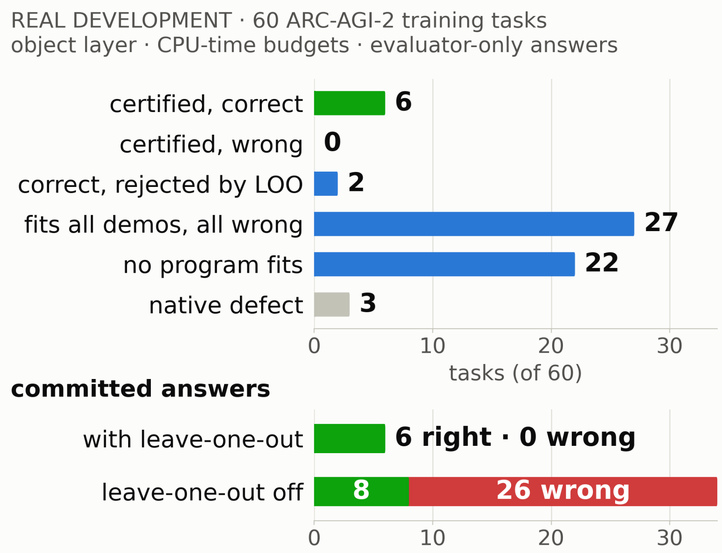
Four achievements, four statuses

achievement	status
detect certain reasoning failures	built, with limits
construct and verify an executable correction	one synthetic and one exposed real task; 0 of 58 on an independent set
learn to choose better revisions from failures	**not established**
transfer a revision policy to biology	**not tested**

The message

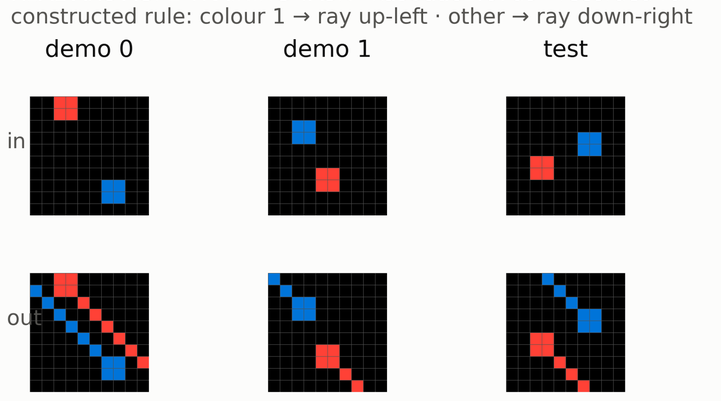
Intelligence may require more than finding an answer. It may require recognizing when the concepts and rules used to build the answer are inadequate, and deciding what must change.

Where CORA fails on real tasks DONE



Object reasoner, 60 exposed training tasks. Figure: run without the frozen assets (27 all-wrong fits; check off, 8 right and 26 wrong). As submitted, with the assets: 26, and 8 right and 25 wrong. In both, the check accepted 6 answers, all correct, and a correct candidate existed on 8. This is one branch, not a reliable solver.

Building a missing rule, verified DONE, NARROW



committed answer: correct (failure-blind arm: same)

Real training task. Frozen CORA and extra search both fail. CORA built a rule that says *where* it applies as well as *what* it does: colour-1 objects shoot a ray north-west, others south-east. It was installed reversibly, rebuilt in 3 of 3 leave-one-out folds, and answered correctly. **Limits:** the failure autopsy selected the task; it is an exposed training task; a failure-blind constructor found the same rule.

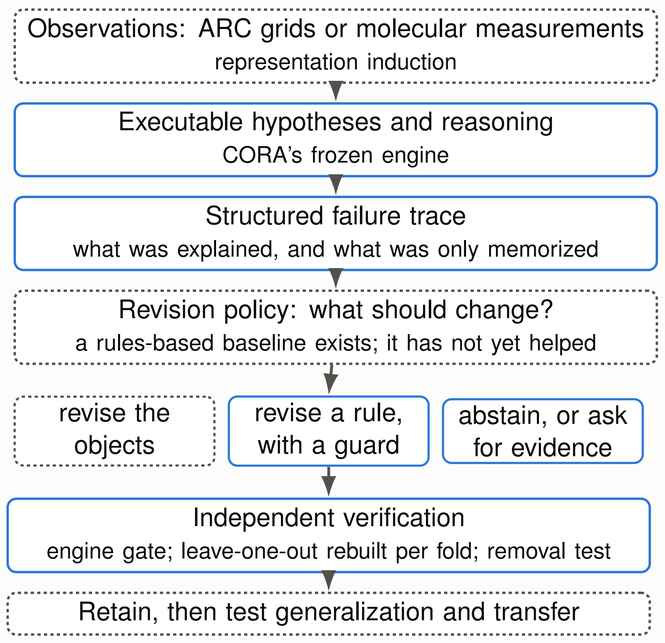
Synthetic task: a single-step ring around the minority-colour cell, 3 of 3 folds, correct.

Does learning from the failure help? NOT ESTABLISHED

- A frozen 10-case pilot: failure record 2 correct, failure-blind 1 correct plus 1 tie abstention, 0 wrong, 0 lost. One discordant pair ($p = 0.5$) is not evidence.
- A hand-designed controller that picks the kind of revision missed the only task that extra search fixed: 6 against 7.
- Why construction rarely fires: of 62 eligible tasks, 20 left no evidence and 39 had evidence no rule fitted. Of 25 such real cases, 18 were inexpressible in the grammar.

Constructing a rule and learning which revision to make are different capabilities.

A revision architecture solid: built · dashed: proposed



An independent test NEGATIVE

58 training tasks chosen by a fixed rule, with the protocol frozen before the run. No construction arm produced a certified answer: learning from the failure, failure-blind, shuffled, and a new arm that learns *where* from the boundary between explained and memorized objects all scored 0 of 58, with 0 wrong. Frozen CORA solved 1. The real-task solve did not generalize, and the failure record showed no advantage. A forensic check that reads the answers shows why: 56 of the 58 need operations the grammar cannot express. The language's reach is the bottleneck, which is the grammar-induction question.

Scientific prediction PROPOSED, NOT TESTED

With molecular measurements, should the system revise its representation, revise its hypothesis, or ask for an experiment? What could transfer is the revision policy and the verification interface, not ARC operators. A biological check needs tolerance, real replicates, held-out conditions and experiments, not exact match. The problem contract (questions for Pierce and Toni) is in review; no biological result is claimed.

Benchmark context

Official ARC-AGI-2 public scores: native CORA **0.00**; a reproduction of the published NVARC Qwen3-4B pipeline **27.64**, which is **not CORA**. The CORA-FHR research entry (18 October) scored **0.00**, as its local rehearsal predicted (no new answer committed). The guarded rule came later and is not part of it.

The question I want to pursue

Can a reasoning system learn from the boundary between what its current knowledge explains and what it cannot, decide what must change, and acquire a capability that generalizes? ARC offers a controlled place to find out before testing it in science.

Every number, with its scope and hash: `records/CORA_SUMMIT_CLAIM_LEDGER_20261017.md`. Authorship and credits are pending collaborator review.